Multi-Objective Optimization for Large-Scale Neural Solver Deployment for Combinatorial Optimization

Yuxuan LI
Supervisor: Ruibin BAI

Abstract. Neural combinatorial optimization (NCO) reports solution quality at a single operating point, leaving system-level questions unanswered: where inference time goes, how much of it is recoverable, and whether deployment is worth its cost. This proposal recasts "large-scale" in NCO as a *deployment-scale* property — instances served, concurrency, and sustained operating duration — rather than instance size alone. Work proceeds in four layers: (A) characterization of where inference cost is spent and how hotspots migrate with scale; (B) quantification of recoverable headroom from compilation, batching, and precision; (C) a multi-objective formulation yielding Pareto-optimal operating points and a deployment decision rule under a quality SLA; and (D) a generalization study across problem family and execution backend. A central methodological contribution is a *hardware calibration anchor* that converts cross-backend latency ratios into a normalized efficiency metric, separating three findings a naive comparison conflates: the hardware is slower, the backend is inefficient, or the workload is bound by neither and dominated by launch and synchronization overhead.

1 Introduction

Neural combinatorial optimization (NCO) has matured into a rich landscape of learned solvers: autoregressive construction [8, 9], heatmap prediction with search [7, 14], improvement and local search [6, 12], and decomposition for large instances [4, 17]. Reporting conventions have not kept pace. Papers typically give one (quality, runtime) point per instance size, on one accelerator, with runtime as an undifferentiated wall clock. Three questions that decide whether such a solver can actually be deployed are therefore unanswered.

Where does the time go? Autoregressive decoders execute a serial loop of $O(n)$ steps, each dispatching many small kernels and — in most implementations — synchronizing with the host to test a termination condition. The relative cost of launch, synchronization, and Python dispatch against genuine arithmetic is strongly regime-dependent, so the bottleneck is expected to *move* with scale. This migration has not been systematically mapped.

How much is recoverable? Compilation, fusion, batching, and reduced precision are standard remedies, but their headroom in NCO is unquantified. Some of the largest overheads are structurally invisible to the standard remedy: `torch.compile` cannot remove a synchronization originating in Python control flow outside the compiled subgraph.

Is it worth it? The Amortized Efficiency Threshold (AET) framing [1] asks how many instances a neural solver must serve before its training energy is amortized by its per-instance inference advantage, reporting a crossover near 1.58×10^5 instances at n=20 — validated, by its authors' account, at a single scale. Whether the threshold rises or inverts with scale, batch, and hardware is open.

The stakes are concrete. On TSP-1000, POMO with 8-fold augmentation is reported at 40.57% gap and 8.5 minutes, against 3.168% and 1.6 minutes for a greedy heatmap-plus-search method [11]. The quality–latency–scale trade-off is thus the dominant design question, and it is most acute in vehicle routing, where deployment pressure is highest and where the gap between what learning-based methods promise and what practitioners can operate remains a recognized open problem [2].

A substantial efficiency literature exists: StruDiCO reports 3.5× speedup and 70% memory reduction, and Fast T2T trades one-step generation for order-of-magnitude gains [10, 15]. Both are *algorithmic* — they change the sampler to reduce work. This proposal is complementary and deliberately *system-level*: it proposes no new solver and does not compete on quality. It asks what a fixed, representative solver family costs to run at deployment scale, what fraction of that is

recoverable by systems techniques alone, and how the answer changes when the deployment target changes.

Contributions. (i) A reproducible characterization of where NCO inference time is spent and how the hotspot distribution migrates with scale. (ii) A quantified account of recoverable headroom, including explicit treatment of the synchronization bottleneck that compilation cannot address. (iii) A multi-objective deployment formulation yielding Pareto-optimal points, hypervolume comparison, and a decision rule under a quality SLA. (iv) A generalization study across problem family and execution backend, with a hardware calibration anchor that makes cross-backend comparison interpretable rather than merely descriptive.

2 Scope and Operational Definitions

2.1 What "Large-Scale" Means Here

In the NCO literature, "large-scale" almost always means large *instance size* n. For a deployment question the binding constraint is usually else: how many instances arrive, how many are served concurrently, and how long the service runs uninterrupted. We therefore define **deployment scale** (primary) as the number of instances N, the concurrency B, and the sustained operating duration τ; and **instance scale** (secondary) as per-instance size n, with training to $n \leq 500$ and zero-shot inference generalization to $n = 1000$.

This changes what is measured. Sustained duration becomes a first-class factor, and it is the factor on which heterogeneous deployment targets are most likely to diverge, since it exposes thermal and power behavior that short benchmarks cannot reveal.

2.2 Objectives

The objective vector is $\boldsymbol{f}(x) = (\text{gap}, \text{p95 latency}, \text{peak memory}, E_{\text{inst}})$, evaluated at fixed hardware and batch size. These conflict in the relevant regime: more samples or deeper search improves gap at the cost of latency and energy; larger batches improve throughput but raise peak memory.

One scoping decision matters. Only *quality and latency* define the Pareto front proper. Peak memory is treated as a feasibility constraint — an operating point that does not fit is not a candidate — and energy as a reported attribute feeding the AET analysis. Four-objective fronts over the sample sizes available here are dominated by mutually non-dominated points and carry little information.

2.3 Measured Asymmetries

Two asymmetries between the deployment targets are not fixable and are stated as scope boundaries rather than controlled for. *Energy on Apple silicon:* macOS exposes no per-process GPU energy interface, so energy objectives and the AET analysis are performed on the CUDA/Linux arm only; the cross-backend comparable metric set is latency, throughput, and peak memory. *Accelerator memory:* 8 GB unified versus 11 GB discrete. Batch size is therefore a grid factor, never a matched constant, and memory exhaustion is a reportable boundary rather than a failed run.

3 Research Questions and Hypotheses

Hypotheses predict a *shape* — a crossover, a trend, a reversal — rather than a direction, since shape claims are the ones most likely to survive as contributions when specific numbers do not reproduce.
RQ1 — Characterization. Where is inference time spent, and does the hotspot distribution migrate with n, B, and decoding budget? **H1.** At small n and B cost is dominated by kernel launch, host–device synchronization, and Python dispatch; as these grow, cost migrates to attention and memory bandwidth. Operationally the normalized efficiency E (§4.6) starts well below 1 and approaches 1 with scale. *The crossover scale is itself a primary result*, identifying where the workload becomes compute-bound.
RQ2 — Optimization headroom. What is the benefit of compilation, fusion, batching, and reduced precision, and does it scale with n? **H2.** Benefit increases with n with clear diminishing returns. More specifically: gains from `torch.compile` are *small precisely where H1 predicts overhead dominates*, because the dominant overhead lies outside the compiled subgraph. Converting the variable-length decode loop to a fixed-length loop removes the synchronization and enables graph capture. This predicts an interaction — compile-only helps little at small

scale, loop restructuring helps a great deal — and testing it is a principal objective.

RQ3 — Deployment configuration. Under a quality SLA (gap $\leq \delta$), what is the cost-optimal configuration, and can it be predicted rather than searched? **H3.** Latency and memory are predictable from $(n, B, \text{hardware}, \delta)$ by a fitted cost model whose error is small relative to variation across configurations, making the decision rule usable without exhaustive search.

RQ4 — Amortized efficiency. What is the AET relative to HGS/LKH, and how does it vary with n, B, and hardware? **H4.** AET decreases substantially as n grows, since the batching advantage of neural inference amplifies with scale while per-instance training cost is fixed — a direct extension of [1] beyond its single validated scale.

RQ5 — Generalization. Do the observed laws survive a change of problem family and execution backend, and do optimal configurations transfer? **H5.** The *shape* of the trade-off transfers, but optimal operating points and their ordering do not — reversals occur. Two mechanisms are predicted: sustained-drift crossover, where a thermally constrained target leaves the SLA under long τ while an actively cooled target does not; and synchronization-cost asymmetry, where a backend with higher per-op dispatch cost loses at exactly the small-scale points H1 identifies as overhead-bound.

4 Methodology

4.1 System Under Study

The primary system is RL4CO [3], built on PyTorch, TorchRL, TensorDict, and PyTorch Lightning. At the pinned revision it provides 20 model families and 24 problem environments. Its value here is threefold: it supplies all four paradigms of §4.5 behind one interface; it separates environment from method, so problem and algorithm vary independently; and its decode loop is a representative, *unoptimized* autoregressive implementation, which is what makes it a meaningful object of systems study. The reference workload for Layers A–C is Attention Model on TSP-100.

4.2 Layer A: Profiling and Characterization

Tooling is applied in fixed order, least to most invasive, so observations cross-validate and measurement perturbation is bounded: `py-spy` (Python versus native flame graphs); `scalene` (CPU/accelerator/memory attribution); `perf` (hardware counters, Linux arm); Nsight Systems and `ncu` (kernel timeline, occupancy, and the inter-kernel *idle gaps* that reveal launch and synchronization stalls); and `pynvml` for power traces.

Each configuration records end-to-end latency (p50/p95/p99), peak accelerator memory, throughput, and an encode/decode phase split; sustained runs additionally record a latency drift series. **Output:** flame graphs, a hotspot ranking, a hotspot-migration figure, and the E crossover of H1 — the paper's central characterization result.

4.3 Layer B: System-Level Optimization

Interventions are grouped by priority in Table 1, cheapest and most standard first.

On precision. Autocast semantics and operator coverage differ between CUDA and Metal, and unsupported operations may fall back silently. Reduced precision is therefore a CUDA-side optimization axis, not a cross-backend comparison condition. The comparable cross-backend baseline is uniformly FP32 (`precision=32-true`, `matmul_precision=highest`); any reduced-precision Metal result is reported as exploratory and excluded from cross-backend claims.

4.4 Layer C: Multi-Objective Deployment Evaluation

Following §2.2, the Pareto front is computed over (gap, p95 latency). An ε-constraint sweep imposes gap $\leq \delta$ and minimizes latency, tracing the front per fixed (hardware, batch) slice so that fronts stay two-dimensional and comparable. Hypervolume compares front quality across configurations and backends; because HV depends on objective scaling and on the reference point, both are fixed and reported explicitly — objectives normalized per instance scale, gap expressed against best-known, reference point set at the worst observed value per objective inflated by a fixed margin. Without these choices stated, HV numbers are neither reproducible nor comparable to future work.

The Amortized Efficiency Threshold is

$$\text{AET}_E \;=\; \frac{E_{\text{train}}(M)}{\max\big(E^{\text{b}} - E^{\text{nn}},\; \epsilon\big)}, \qquad (1)$$

subject to $\text{gap}(M, \pi) \leq \text{gap}(B, \pi) + \delta$, where E^{b} is the per-instance energy of the baseline heuristic

P	Intervention	Description and measurement
P0	`torch.compile`	Compile the encoder subgraph. Record guard failures, recompilation count, dynamic-shape penalty.
P0	Batching and padding	Quantify padding waste and the throughput effect of length bucketing.
P0	Reduced precision (AMP)	Sensitivity of gap and energy to precision. *CUDA arm only* — see below.
P1	**Synchronization elimination in the decode loop**	Replace the variable-length termination check with a fixed-length loop, using the fact that decoding length is known a priori for TSP (n steps) and bounded for CVRP. Removes per-step synchronization, makes shapes static, enables graph capture. Measure synchronization count, accelerator idle time, end-to-end latency.
P2	ONNX / ONNX Runtime	Static subgraph export; masking and dynamic control flow complications documented rather than worked around.
P3	TensorRT / INT8	Conditional. The reference CUDA device is compute capability 7.5, lacking reduced-precision hardware paths and excluded from current FlashAttention-2 releases. A negative result is expected and acceptable.

Table 1: System-level interventions by priority. P1 removes a bottleneck that compilation cannot reach.

and E^{nn} that of the neural solver under policy π. Crucially $E_{\text{train}}(M)$ includes all failed runs and hyperparameter search, not only the converged model; excluding them understates the numerator by one to two orders of magnitude and is the most common way AET figures mislead.

Output: a reproducible front with ≥ 5 points, an HV comparison across configurations, and an AET surface over n, B, and hardware. The practical deliverable is a *deployment decision rule*: given a workload shape and a quality SLA, which configuration to run, and whether a neural solver is justified at all against a strong heuristic.

4.5 Layer D: Generalization

Cross-backend (not cross-architecture). This axis compares CUDA on a discrete accelerator against Metal/MPS on Apple silicon. **The comparison must be described as cross-backend, not cross-architecture.** The two systems differ simultaneously in instruction set (x86-64 versus ARM64), accelerator API (CUDA versus Metal), memory system (discrete GDDR6 at ∼616 GB/s versus unified LPDDR5 shared with the OS), thermal design (actively cooled versus fanless), and runtime stack (CUDA runtime versus Metal Performance Shaders). No observed difference can be attributed to processor architecture.

What this axis *can* establish: the existence and location of portability boundaries, the cost of operator fallback, and whether optimal operating points transfer. What it *cannot*: any claim about ARM versus x86. A boundary result is an expected contribution here, framed as a portability finding. A genuine architecture claim requires a CUDA-capable ARM target, which isolates instruction set from accelerator API; a Jetson Orin device is requested in §6 and, if obtained, converts this into a controlled study.

Cross-problem. Transfer from TSP to CVRP, which adds a capacity constraint. Training at $n \in \{50, 100\}$; evaluation at $n \in \{200, 500, 1000\}$ is strictly zero-shot, since fine-tuning at test scale would dissolve the question. The criterion is whether the shape of the trade-off and the ordering of configurations are preserved under the added constraint.

Cross-model. Generalization across solver structure is tested *within* the reference codebase, spanning all four paradigms: autoregressive (`am`, `pomo`, `symnco`, `mdam`), non-autoregressive heatmap (`nargnn`, `l2d`), improvement (`dact`, `neuopt`, `eas`), and decomposition (`glop`). This is deliberate. Comparing against an independently developed solver introduces a confound that is easy to overlook: a second codebase brings its own data pipeline, batching implementation, and instrumentation points, so a measured difference is a difference between two *implementations*, not two *architectures*, and the harness itself becomes an uncontrolled variable. Holding the harness fixed isolates the quantity of interest. Where an external solver is available it serves as an external validation point — a law established here is re-

E	Regime	Interpretation
≈ 1	Compute-bound	Time ratio tracks silicon; the backend is a faithful, slower implementation.
$\ll 1$	Overhead-bound	Time is spent outside arithmetic, so latency is similar despite different hardware. *Evidence for H1.*
$\gg 1$	Backend inefficiency	The target leaves hardware unused — operator fallback, per-op synchronization. A portability finding, not a hardware property.

Table 2: Interpretation of the normalized efficiency metric E.

tested on it — which is a strengthening result if obtained and not a dependency if not.

4.6 Hardware Calibration Anchor

A raw cross-backend latency ratio is not interpretable on its own. If a workload takes 4.6× longer on backend t than on reference r, that is *equally consistent* with three different findings: the target hardware is slower, the target backend is inefficient, or neither machine is doing arithmetic and both are bound by launch and synchronization overhead. We therefore establish a calibration anchor — sustained FP32 matrix multiplication throughput, measured identically on each target (a 4096^3 matmul, synchronized on both sides of the timed region). This is the most vendor-optimized operation available and bounds achievable compute rate from above; it is not a predictor of solver performance.

Let Π_r, Π_t be calibrated throughputs. For workload w,

$$H_t = \frac{\Pi_r}{\Pi_t}, \quad R_{t,w} = \frac{T_{t,w}}{T_{r,w}}, \quad E_{t,w} = \frac{R_{t,w}}{H_t}. \quad (2)$$

Table 2 reads this metric.

H1 thereby takes a sharp form: *E should rise from $\ll 1$ at small scale toward 1 as n and B grow*, and the crossover scale is reportable. The interesting question is not the absolute gap between deployments but whether E moves under optimization, whether it moves differently at different scales, and where it crosses the deployment SLA.

5 Experimental Protocol

5.1 Factors and Design

A full factorial over all factors is neither feasible nor statistically useful for a characterization study. Layer A runs the full $n \times B$ grid (16 cells) with decoding fixed, plus a sustained-duration series at anchors; decoding budget and precision are varied one-at-a-time at a few anchor configurations. Layer B varies interventions one at a time against the eager baseline on a reduced n grid, so effects are attributable. Layer C uses the full decoding $\times$ δ product, which ε-constraint sweeping requires. Table 3 lists the factors and the level at which each is varied.

Decoding budget is evaluated at three interpretable tiers — *fast* (greedy or single sample), *balanced* (fixed sample count), *quality* (fixed time or search depth) — yielding a decision curve rather than a point estimate.

5.2 Instance Sets and Determinism

Instance sets are generated once with fixed seeds using the reference codebase's public generators, serialized to disk, and loaded by every subsequent run. They are *never* regenerated per run and *never* regenerated per backend: a cross-backend comparison on independently sampled instances compares different problems. External validity uses a TSPLIB subset and CVRPLIB Set-X against official optima or best-known solutions. Datasets, checkpoints, and logs are written outside any cloud-synchronized directory — background synchronization during a timed run perturbs wall-clock measurements invisibly.

5.3 Measurement and Reproducibility

Fixed warm-up (≥ 3 iterations) discards first-call compilation and clock ramp-up; ≥ 30 repetitions per configuration give mean, standard deviation, and 95% confidence intervals. Comparisons are *paired* on the same instance set and seed across arms, using common random numbers to reduce variance. Instance-level and run-level variability are reported *separately*: thirty repetitions of the same 100 instances are not 300 independent samples for instance-level effects. The accelerator is synchronized before reading the clock — CUDA and MPS both queue asynchronously, so without synchronization one measures enqueue, not compute. Energy is integrated as $\int P\,dt$, not

Factor	Levels	Full grid	Notes
Problem	TSP, CVRP	Layer D	
Instance size n	50, 100, 500, 1000	Layer A	train ≤ 500; 1000 zero-shot
Batch size B	1, 8, 32, 128	Layer A	grid factor, never matched across arms
Sustained duration τ	short, 5, 15, 30 min	Layer A (anchors)	thermal drift
Decoding budget	fast / balanced / quality	Layer C	greedy, fixed-k, multi-start
Execution mode	eager, compiled, restructured	Layer B	OAT against eager
Precision	FP32, FP16 (AMP)	Layer B	**CUDA arm only**
Hardware target	CUDA, Metal/MPS	Layer D	(+ Jetson if available)

Table 3: Experimental factors. "OAT": varied one-at-a-time at anchor configurations.

averaged power. Clocks are locked where possible; otherwise the full dynamic frequency trace is recorded. For sustained runs the calibration anchor is re-measured before and after, since a thermally constrained target throttles and the throttled throughput would otherwise be silently attributed to the backend.

Reproducibility is a deliverable, not an afterthought. The codebase is pinned by revision hash and held read-only; environment problems are resolved in the environment, never by editing source, since a measurement must correspond to a specific unmodified revision. Each machine produces a machine-generated environment record (interpreter and package versions, revision, hardware capability, calibration anchor), and every table and figure is regenerated by script from raw measurements — no number is transcribed by hand.

5.4 Metrics

Optimality gap: $(c_{\text{model}} - c_{\text{ref}})/c_{\text{ref}}$, reported as mean, quantiles, and failure count. Where gap saturates at zero (small n, strong solvers) we report the fraction of instances solved to best-known rather than a degenerate zero. **Latency**: end-to-end p50/p95/p99, split into encode and decode. **Throughput**: instances/s, and samples/s during sampling, at fixed n so large instances are not confounded with low counts. **Peak memory**: maximum accelerator memory allocated, reset per configuration. **Energy**: J/instance and J per unit gap improvement, CUDA/Linux arm only. **Training cost**: wall-clock per epoch and total cost to reach target gap, with optimizer, batch size, and parallel environment count reported. **Deployment cost**: AET in instances, with ε and grid carbon intensity reported.

Reference solutions come from LKH-3 for TSP [5] and PyVRP for CVRP [16], the latter also being the baseline used by the AET framework, preserving comparability with that line of work. OR-Tools [13] is used for independent feasibility checking. An exact solver (Gurobi, academic license held) is viable for certified-optimal references only at small n and for tightly constrained CVRP; it is *not* a viable gap denominator for TSP at $n \geq 500$, where it cannot close the gap in practical time.

6 Resources, Timeline, and Risks

6.1 Hardware, Limits, and Requests

The available hardware is an RTX 2080 Ti (11 GB, Turing, sm_75) and a MacBook Air M2 (8 GB unified, fanless). Verified properties matter here: the 2080 Ti executes native compiled code with no PTX JIT fallback, but 11 GB constrains n=500 training with multi-start decoding, it has no reduced-precision hardware path, and it is excluded from current FlashAttention-2 releases. The M2 has partial operator coverage with silent CPU fallback, cannot represent edge ARM deployment generally, exposes no per-process accelerator energy, and throttles under sustained load.

Note the asymmetry this creates. Jointly the two devices suit Layers A, B, and the cross-backend part of D well — and the thermal contrast between an actively cooled and a fanless device is, unusually, an *asset* for the sustained-duration factor. They are not suited to large-scale training, symmetric energy measurement, or any

instruction-set architecture claim.

Three requests follow, in order of necessity. **An Ubuntu installation or dual-boot on the CUDA machine** — RQ4 requires energy measurement via RAPL, which has no supported interface on Windows; this is a hard prerequisite for RQ4, not a convenience. **An NVIDIA Jetson Orin or ARM server** — the only way to isolate instruction set from accelerator API; without it the architecture axis remains a backend comparison. **A GPU with ≥ 24 GB or A100 node access** — unlocks n=500 training and the larger batch sizes. Helpful: PC memory to 32 GB for n=1000 dataset generation.

6.2 Timeline

The study is designed so that a defensible contribution exists at several exit points, not only at the end. The **minimum viable result**, achievable on current hardware, is: reproducible profiling and benchmarking code; a characterized hotspot migration with the E crossover; measured optimization headroom; ≥ 5 Pareto-optimal points with hypervolume comparison; and an honest record of negative results. The plan in Table 4 is sequenced to protect that result.

6.3 Risks

Each risk carries a pre-committed mitigation and an early-warning signal, so the response is decided before the signal appears rather than after it. The signal column exists so that a risk is detected when it first becomes true, not when it becomes fatal. Table 5 lists them.

Month	Phase	Deliverable	Go/No-go
M1	Environment freeze, baseline	Pinned environment records per machine; RL4CO TSP-50/100 running; LKH-3, PyVRP, OR-Tools references; calibration anchor on both targets.	Reproduced gap matches published magnitude; anchors reproducible to a few percent.
M2	Characterization (A)	Flame graphs, hotspot ranking, $n \times B$ grid, sustained-duration series, hotspot-migration figure, E crossover.	Migration visible; E trend monotone as predicted.
M3	Optimization I (B, P0)	Compile, batching, precision ablations against eager.	Attribution clean: one intervention at a time.
M4	Optimization II and multi-objective (B P1, C)	Decode-loop synchronization elimination with graph capture; ε-constraint sweep; Pareto front and hypervolume; decision rule.	≥ 5 Pareto points; *minimum viable result complete*.
M5	Deployment cost and cross-model (RQ4)	AET computation and sensitivity; cross-paradigm generalization; external validation if available.	AET surface over n, B, hardware; Linux environment in place.
M6	Cross-backend, cross-problem, writing (D)	TSP to CVRP transfer; backend comparison with reversal analysis; Jetson results if available; draft.	All tables regenerable from scripts end-to-end.

Table 4: Six-month plan; a complete result is targeted at M4.

Risk	Signal	Mitigation
Memory insufficient	OOM at n=500 training	Restrict n=1000 to inference; gradient checkpointing; FP16; smaller batch. *Do not modify model architecture to mask a systems problem.*
`torch.compile` yields little	Speedup $< 5\%$	Expected if H1 holds. Pivot weight to decode-loop restructuring (P1) and the AET cost model, which are the primary contributions regardless. The negative result becomes evidence *for* H1.
Metal not comparable	Substantial operator fallback	Report as a portability boundary. Silent-fallback characterization is a legitimate result and does not invalidate the CUDA-side findings.
RQ4 blocked	No Linux environment by M5	Defer RQ4; Layers A–C do not depend on it.
Scope overrun	Layer B not converged by end of M3	Cut P2 and P3; drop the Jetson arm; retain the two-backend comparison and the complete Pareto analysis. The minimum viable result is intact.
No external solver available	Not obtained by week 4	The cross-paradigm axis within the reference codebase is self-sufficient and, as argued in §4.5, methodologically cleaner. This is the designed fallback, not a degradation.

Table 5: Risks, signals, and pre-committed mitigations.

References

[1] Anonymous. An amortized efficiency threshold for comparing neural and heuristic solvers, 2026. Author list to be completed.

[2] R. Bai, X. Chen, Z.-L. Chen, T. Cui, S. Gong, W. He, X. Jiang, H. Jin, J. Jin, G. Kendall, et al. Analytics and machine learning in vehicle routing research. *International Journal of Production Research*, 61(1):4–30, 2023.

[3] F. Berto, C. Hua, J. Park, L. Luttmann, Y. Ma, F. Bu, J. Wang, H. Ye, M. Kim, S. Choi, et al. RL4CO: an extensive reinforcement learning for combinatorial optimization benchmark, 2023.

[4] L. Duan, J. Wang, Z. Yang, Y. Shi, and D.-C. Zhan. H-TSP: Hierarchically solving the large-scale travelling salesman problem. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2023.

[5] K. Helsgaun. An extension of the Lin–Kernighan–Helsgaun TSP solver for constrained traveling salesman and vehicle routing problems. Technical report, Roskilde University, 2017.

[6] A. Hottung, Y.-D. Kwon, and K. Tierney. Efficient active search for combinatorial optimization problems. In *International Conference on Learning Representations (ICLR)*, 2022.

[7] C. K. Joshi, T. Laurent, and X. Bresson. An efficient graph convolutional network technique for the travelling salesman problem, 2019.

[8] W. Kool, H. van Hoof, and M. Welling. Attention, learn to solve routing problems! In *International Conference on Learning Representations (ICLR)*, 2019.

[9] Y.-D. Kwon, J. Choo, B. Kim, I. Yoon, Y. Gwon, and S. Min. POMO: Policy optimization with multiple optima for reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[10] Y. Li, J. Guo, R. Wang, H. Zha, and J. Yan. Fast T2T: Optimization consistency speeds up diffusion-based training-to-testing solving for combinatorial optimization. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[11] F. Luo, X. Lin, F. Liu, Q. Zhang, and Z. Wang. Instance-conditioned adaptation for large-scale generalization of neural combinatorial optimization, 2024.

[12] Y. Ma, J. Li, Z. Cao, W. Song, L. Zhang, Z. Chen, and J. Tang. Learning to iteratively solve routing problems with dual-aspect collaborative transformer. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

[13] L. Perron and V. Furnon. Or-tools. Google, 2023. URL `https://developers.google.com/optimization`.

[14] Z. Sun and Y. Yang. DIFUSCO: Graph-based diffusion solvers for combinatorial optimization, 2023.

[15] Y. Wang, Y. Li, J. Yan, and Y. Chang. StruDiCO: Structured denoising diffusion with gradient-free inference-stage boosting for memory and time efficient combinatorial optimization. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

[16] N. A. Wouda and L. Lan. PyVRP: A high-performance VRP solver package. *INFORMS Journal on Computing*, 2024.

[17] H. Ye, J. Wang, Z. Liang, Y. Liang, Z. Cao, Y. Li, and F. Li. GLOP: Learning global partition and local construction for solving large-scale routing problems in real-time, 2023.